SIMATS ENGINEERING

ASSESSMENT TOOL 2

Pseudocode Writing Task (Algorithm Design)

COURSE NAME: COMPUTER VISION WITH OPENCV **COURSE CODE:** DSA0204

COURSE OUTCOME COVERED

CO5: Understand video processing - motion computation - 3D vision - geometry. (BTL 6)

Assessment Tool Weightage: 20%

Pseudocode Writing Tasks: Design a clear and structured pseudocode for the given problem by organizing the solution into logical stages of a computer vision pipeline. Ensure proper use of control flow, modular steps, and justification of key operations to satisfy the given constraints.

Code of Conduct:

I K Sampath Reddy (192424285) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____K Sampath Reddy_____

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

4. Vehicle Counting System using Video

A traffic monitoring system must:

A. Read video frames

B. Detect vehicles entering a region of interest (ROI)

C. Track movement across frames

D. Count vehicles and display count

Constraints: ● Avoid double counting ● Handle multiple vehicles simultaneously

Write pseudocode for the complete system, including detection, tracking, counting logic, and visualization using drawing functions.

5. Facial Expression Recognition System

A mobile-based system must:

A. Capture images from a camera

B. Detect faces

C. Extract facial features

D. Classify expressions (happy, sad, neutral)

E. Display results

Constraints: ● Limited computational resources ● Real-time response required

Write pseudocode for the pipeline, ensuring efficient processing and modular design for detection, feature extraction, and classification.

6. Real-Time Object Detection Pipeline

A system must:

A. Capture video frames

B. Detect objects using a trained model

C. Draw bounding boxes and labels

D. Display results in real time

Constraints: ● Must process frames efficiently ● Handle multiple objects

Write detailed pseudocode with modular steps for detection, visualization, and efficient looping.

PSEUDOCODE SOLUTIONS

Question 4: Vehicle Counting System using Video

Overview

Vehicles are detected within a defined region of interest (ROI/counting line), tracked frame-to-frame using centroid tracking, and counted only once when their track crosses the counting line — avoiding double counts even with multiple simultaneous vehicles.

Pseudocode

```
BEGIN VehicleCountingSystem

  MODULE Initialize():
    OPEN video_stream = VideoCapture(source)
    vehicle_detector = LOAD detection_model        // e.g. background subtraction or trained detector
    DEFINE ROI_line = (y = LINE_Y)                 // horizontal counting line
    tracked_vehicles = EMPTY LIST
    vehicle_count = 0
    RETURN video_stream, vehicle_detector

  MODULE DetectVehicles(frame, vehicle_detector):
    fg_mask = vehicle_detector.Apply(frame)
    fg_mask = Morphology_Open(fg_mask)
    fg_mask = Morphology_Dilate(fg_mask)
    contours = FindContours(fg_mask)
    detections = EMPTY LIST
    FOR EACH contour IN contours DO
      IF ContourArea(contour) > MIN_VEHICLE_AREA THEN
        detections.APPEND(BoundingRect(contour))
      END IF
    END FOR
    RETURN detections

  MODULE TrackVehicles(tracked_vehicles, detections):
    FOR EACH detection IN detections DO
      match = FindNearestTrack(tracked_vehicles, detection)
      IF match FOUND THEN
        match.UpdatePosition(detection)
      ELSE
        new_track = CreateTrack(detection, id = GenerateID(), counted = FALSE)
        tracked_vehicles.APPEND(new_track)
      END IF
    END FOR
    REMOVE tracks WHERE track.missed_frames > MAX_MISSED
    RETURN tracked_vehicles

  MODULE CountCrossings(tracked_vehicles, ROI_line):
    count_increment = 0
    FOR EACH track IN tracked_vehicles DO
      IF track.counted == FALSE AND track.HasCrossed(ROI_line) THEN
```

```
            track.counted = TRUE            // mark to avoid double counting
            count_increment = count_increment + 1
        END IF
    END FOR
    RETURN count_increment

// ---------------- MAIN LOOP ----------------
(video_stream, vehicle_detector) = Initialize()

WHILE video_stream.IsOpened() DO
    SUCCESS, frame = video_stream.ReadFrame()
    IF NOT SUCCESS THEN
        BREAK
    END IF

    detections = DetectVehicles(frame, vehicle_detector)
    tracked_vehicles = TrackVehicles(tracked_vehicles, detections)
    vehicle_count = vehicle_count + CountCrossings(tracked_vehicles, ROI_line)

    DrawLine(frame, ROI_line, color = YELLOW)
    FOR EACH track IN tracked_vehicles DO
        DrawRectangle(frame, track.bounding_box, color = BLUE)
    END FOR
    PutText(frame, "Count: " + vehicle_count, position = (10, 30))

    DisplayFrame(frame)
    IF KeyPressed() == 'q' THEN
        BREAK
    END IF
END WHILE

video_stream.Release()
CloseAllDisplayWindows()
END
```

Justification of Key Operations

- Each track carries a persistent 'counted' flag; once TRUE it is never re-counted, directly preventing double counting.
- Centroid-based tracking (rather than per-frame detection counting) preserves vehicle identity across frames so the same vehicle is not treated as a new object.
- Processing detections inside a FOR EACH loop naturally supports multiple simultaneous vehicles.
- A single counting line (ROI) with a crossing test keeps the counting logic O(n) per frame, suitable for real-time traffic monitoring.

Question 5: Facial Expression Recognition System

Overview

Given limited computational resources on a mobile device, the pipeline uses a lightweight cascade for face detection, extracts a compact set of facial landmarks/features, and classifies expression with a small pre-trained classifier — minimizing per-frame cost to stay real-time.

Pseudocode

```
BEGIN FacialExpressionRecognition

 MODULE Initialize():
    LOAD lightweight_face_detector         // e.g. Haar Cascade for low compute cost
    LOAD landmark_model                    // compact facial landmark predictor
    LOAD expression_classifier             // lightweight trained classifier
    OPEN camera_stream = VideoCapture(source = 0)
    RETURN camera_stream

 MODULE CaptureAndPreprocess(camera_stream):
    frame = camera_stream.ReadFrame()
    resized_frame = Resize(frame, target_size = SMALL_SIZE)   // reduce compute
    gray_frame = ConvertToGrayscale(resized_frame)
    RETURN frame, gray_frame

 MODULE DetectFace(gray_frame):
    faces = lightweight_face_detector.DetectMultiScale(gray_frame)
    RETURN faces

 MODULE ExtractFeatures(gray_frame, face_rect):
    (x, y, w, h) = face_rect
    face_roi = CROP(gray_frame, x, y, w, h)
    landmarks = landmark_model.Predict(face_roi)    // key points: eyes, brows, mouth
    feature_vector = ComputeGeometricFeatures(landmarks)  // distances/angles
    RETURN feature_vector

 MODULE ClassifyExpression(feature_vector):
    expression = expression_classifier.Predict(feature_vector)  // happy/sad/neutral
    RETURN expression

 // ---------------- MAIN LOOP ----------------
 camera_stream = Initialize()

 WHILE camera_stream.IsOpened() DO
    (frame, gray_frame) = CaptureAndPreprocess(camera_stream)
    faces = DetectFace(gray_frame)

    IF faces IS EMPTY THEN
       DisplayFrame(frame)
       CONTINUE                    // skip remaining stages, save compute
    END IF
```

```
    face_rect = faces[0]                // primary face for mobile use case
    feature_vector = ExtractFeatures(gray_frame, face_rect)
    expression = ClassifyExpression(feature_vector)

    (x, y, w, h) = face_rect
    DrawRectangle(frame, (x, y), (x+w, y+h), color = GREEN)
    PutText(frame, expression, position = (x, y-10))

    DisplayFrame(frame)
    IF KeyPressed() == 'q' THEN
      BREAK
    END IF
  END WHILE

  camera_stream.Release()
  CloseAllDisplayWindows()
END
```

Justification of Key Operations

- Down-scaling the frame before processing and using a lightweight cascade detector reduce computational load, addressing the limited-resources constraint.
- An early CONTINUE when no face is found avoids running feature extraction/classification unnecessarily, saving cycles for real-time response.
- Geometric (distance/angle) features from a compact landmark set are cheaper to compute than deep-CNN features, fitting mobile constraints while remaining discriminative for basic expressions.
- Modular separation of detection, extraction, and classification allows any stage to be swapped for a lighter/heavier model depending on device capability.

Question 6: Real-Time Object Detection Pipeline

Overview

Frames are captured continuously and passed through a trained object detection model (e.g. YOLO/SSD); resulting bounding boxes and class labels for all detected objects are drawn and displayed each iteration with minimal per-frame overhead.

Pseudocode

```
BEGIN RealTimeObjectDetection

  MODULE Initialize():
     LOAD trained_object_detection_model    // e.g. YOLO / SSD
     OPEN camera_stream = VideoCapture(source = 0)
     RETURN camera_stream

  MODULE PreprocessFrame(frame):
     resized = Resize(frame, model_input_size)
     normalized = Normalize(resized)
     RETURN normalized

  MODULE DetectObjects(preprocessed_frame, model):
     raw_predictions = model.Forward(preprocessed_frame)
     detections = NonMaxSuppression(raw_predictions, iou_threshold = 0.45,
                                    confidence_threshold = 0.5)
     RETURN detections                    // list of (box, class_label, score)

  MODULE DrawDetections(frame, detections):
     FOR EACH detection IN detections DO      // handles multiple objects
        (box, class_label, score) = detection
        DrawRectangle(frame, box, color = ColorForClass(class_label), thickness = 2)
        PutText(frame, class_label + " " + FORMAT(score), position = box.top_left)
     END FOR
     RETURN frame

  // ---------------- MAIN LOOP ----------------
  camera_stream = Initialize()

  WHILE camera_stream.IsOpened() DO
     SUCCESS, frame = camera_stream.ReadFrame()
     IF NOT SUCCESS THEN
        BREAK
     END IF

     preprocessed = PreprocessFrame(frame)
     detections = DetectObjects(preprocessed, trained_object_detection_model)
     frame = DrawDetections(frame, detections)

     DisplayFrame(frame)
     IF KeyPressed() == 'q' THEN
        BREAK
```

```
    END IF
  END WHILE

  camera_stream.Release()
  CloseAllDisplayWindows()
END
```

Justification of Key Operations

- Resizing/normalizing to the model's expected input size avoids redundant internal resampling, keeping per-frame inference efficient.
- Non-Max Suppression removes duplicate/overlapping boxes for the same object, improving accuracy without extra manual logic.
- Iterating detections in a single FOR EACH loop naturally handles any number of simultaneous objects.
- A single-pass WHILE loop with no blocking calls keeps the display responsive for real-time viewing.